The Diophantine equation $x^2 = n!! \cdot \mathrm{lcm}(1, \ldots, n)$: the complete solution set, formalized and machine-checked in Lean 4

Aurora0134*

October 7, 2026

Abstract

We determine the complete solution set of the Diophantine equation $x^2 = n!! \cdot \mathrm{lcm}(1, \ldots, n)$, where $n!!$ is the double factorial and $\mathrm{lcm}(1, \ldots, n)$ the least common multiple of the integers from 1 to n: for $n \in \mathbb{N}$ the product is a perfect square if and only if $n \in \{0, 1, 2, 5, 7\}$, with square roots $x = 1, 1, 2, 30, 210$. This settles the second of the two cases that Novaković (2026) recorded as believed-finite with the words "at the moment we were not able to give an argument"; the first case, $x^2 = n! \cdot n\#$, was settled separately [9]. The answer given here is an if-and-only-if characterization and is therefore strictly stronger than the finiteness being asked about. The proof combines a valuation criterion ($n!! \cdot \mathrm{lcm}(1, \ldots, n)$ is a square if and only if $\nu_p(n!!) + \lfloor \log_p n \rfloor$ is even for every prime p), an exclusion of even n by Bertrand's postulate, an exclusion of odd $n \geq 33$ by an interval-prime lemma for $(n/4, n/3]$ proved in this work as a prime-certificate chain plus a factorial-ratio tail estimate, and explicit non-square witnesses for the remaining band. Every statement is machine-checked in Lean 4 against mathlib v4.34.0: the statements were frozen by hash before proving, the proof compiles with no `sorry`, and `#print axioms` reports only {propext, `Quot.sound`, `Classical.choice`}. The results were developed with AI assistance under human direction; the Lean 4 kernel is the sole adjudicator, and the disclosure section states the terms. Lean sources, frozen statements, and audit logs accompany the paper as a deposited artifact, and a companion claim of record is deposited separately.

*Contact: via the GitHub account Aurora0134.

1 Introduction

The equation of the title asks when the product of the double factorial and the least common multiple of the initial segment is a perfect square. It appears, together with the companion equation $x^2 = n! \cdot n\#$, in Novaković's 2026 study [1] of the Diophantine equation $P(x) = \prod_{i=1}^{r} H_{n_i}$, where the author writes (verbatim wording; mathematical notation retypeset): "We believe that finitely many solutions occur for $x^2 = n! \cdot n\#$ or $x^2 = n!! \cdot [1, \ldots, n]$, but at the moment we were not able to give an argument". In that paper's notation $[1, \ldots, n]$ denotes $\prod_{p \le n} p^{\lfloor \log_p n \rfloor}$, which is exactly $\operatorname{lcm}(1, \ldots, n)$. The present paper answers the second of the two cases completely: the equation $x^2 = n!! \cdot \operatorname{lcm}(1, \ldots, n)$ has, for $n \in \mathbb{N}$, exactly the solutions $n \in \{0, 1, 2, 5, 7\}$. The first case is not claimed here; it was settled separately [9].

Three contributions are recorded together. First, the mathematical result itself, as an if-and-only-if characterization with explicit witnesses (Section 4); an exclusivity review found no occupying record for this statement on the channels it could reach (Section 7). Second, a complete machine-checked formalization: three statements frozen by hash before proving, proved in Lean 4 against mathlib with no placeholder and with axiom dependencies exactly the three standard ones (Section 6). Third, the artifact is built so that a reader can re-run the adjudication — the frozen statement's hash, the strict compile's exit status and the axiom printout all ship with the deposit — rather than take the text's word for it. This is why the paper addresses a formal-methods readership as well as a number-theoretic one.

Notation

We write $n!!$ for the double factorial ($n!! = \prod_{1 \le k \le n,\ k \equiv n \ (2)} k$, so $7!! = 105$ and $0!! = 1$ by the empty-product convention) and $\operatorname{lcm}(1, \ldots, n)$ for the least common multiple of the integers $1, \ldots, n$ (so $\operatorname{lcm}(1, \ldots, 0) = 1$); $\nu_p(m)$ denotes the exponent of the prime p in m, and $\lfloor \log_p n \rfloor$ the largest k with $p^k \le n$. In the formalization the double factorial is `Nat.doubleFactorial` (notation `n!!`), the least common multiple is `Nat.lcmUpto`, and $\nu_p(m)$ is `Nat.factorization`. Division in the interval-prime lemma (Lemma 6) is natural-number division, so the interval reads $\lfloor n/4 \rfloor < p \le \lfloor n/3 \rfloor$; for prime existence this coincides with the real reading, since p is an integer.

2 Mathematical background

Definition 1. For $n \in \mathbb{N}$, the *double factorial* $n!!$ is the product of the integers $k \leq n$ with $k \equiv n \pmod 2$ (so $0!! = 1!! = 1$ and $5!! = 15$). For a prime p and $m \geq 1$, $\nu_p(m)$ is the exponent of p in the prime factorization of m.

Two classical ingredients enter. The first is Legendre's formula $\nu_p(n!) = \sum_{i \geq 1} \lfloor n/p^i \rfloor$, which reaches the proof through mathlib's `Nat.factorization_factorial` together with the two identities $n! = n!! \cdot (2m)!! = n!! \cdot 2^m \cdot m!$ for odd $n = 2m + 1$ (`Nat.factorial_eq_mul_doubleFactorial`, `Nat.doubleFactorial_two_mul`). The second is Bertrand's postulate (`Nat.bertrand`). The shape of the argument is: a natural number is a square if and only if all its prime valuations are even (`Nat.isSquare_iff_even_factorization`), and for the product at hand the total exponent at p is $\nu_p(n!!) + \lfloor \log_p n \rfloor$, because $\operatorname{lcm}(1, \ldots, n)$ contributes exactly $\lfloor \log_p n \rfloor$ (`Nat.factorization_lcmUpto`). So it suffices to find, for each n outside $\{0, 1, 2, 5, 7\}$, a prime p at which that total is odd. Even n are killed by Bertrand; odd $n \geq 33$ by an interval prime in $(n/4, n/3]$; the remaining band is decided pointwise.

Where this sits in the literature. The equation is the second case of the remark in [1]; the first case $x^2 = n! \cdot n\#$ was settled separately [9], and its interval-prime lemma (for $(n/3, n/2]$) is the structural model for Lemma 6 below, with the same certificate-chain-plus-tail architecture. The same author's other papers [2, 3, 4, 5] are recorded with the restricted wording that their objects do not contain this equation (each was read in full; see Section 7). Adjacent records that must be distinguished from the present statement — the squarefree part of $\operatorname{lcm}(1, \ldots, n)$ (OEIS A056839), the value $(p_n\#)^2$ (OEIS A061742), the squarefree part of $n!$ (OEIS A055204), the disproved Erdős problem #399 [8], primorial-core Pell equations [7], and square products of factorials [6] — are delimited one by one in Section 7.

3 Formalisation setup

The formalization is written against Lean 4 [10], version `v4.34.0` (the `leanprover/lean4` release of that version), with mathlib4 [11] `v4.34.0` at revision `5ed29652`; the pin ships with the artifact (`proofs/lean-toolchain`). The three statements of Listing 1 were frozen by hash before proving began; the proving stage may replace proof bodies

only, and the final audit re-compares the delivered statement headers against the frozen snapshot under two whitespace conventions. Two results the final development consumes come from an accompanying library rather than from mathlib: the interval-prime lemma of Lemma 6 and one bridge used by the witness layer. Their module names appear among the imports of the shipped final file; they are not part of this deposit (Section 8(b)).

Encoding decisions. Four encoding choices are worth recording. (i) The double factorial is mathlib's `Nat.doubleFactorial` and the least common multiple is `Nat.lcmUpto`; the latter's factorization identity $\nu_p(\mathrm{lcm}(1,\ldots,n)) = \lfloor \log_p n \rfloor$ is `Nat.factorization_lcmUpto`, which is what makes the valuation criterion a one-line consequence of the product law. (ii) The main result is delivered in two forms: the iff form `dfact_lcm_square_solution_set` ("the product is a square if and only if $n \in \{0,1,2,5,7\}$") and the existence-witness form `exists_sq_dfact_lcm_iff` ("$x^2 = n!! \cdot \mathrm{lcm}(1,\ldots,n)$ is solvable if and only if $n \in \{0,1,2,5,7\}$"), bridged by `isSquare_iff_exists_sq`; the solution set is written as a disjunction chain, which on $\mathbb{N}$ is equivalent to membership in a finite set. (iii) The interval-prime lemma is stated with natural-number division, faithful to the real reading for prime existence as recorded in the formalization's fidelity note. (iv) The interval-prime lemma is proved in this work rather than cited: it is a finite certificate chain covering a bounded segment plus a factorial-ratio estimate for the tail, and it is not an instance of any published interval theorem (Section 8(b)).

Audit mechanism. The statement is frozen by hash; a strict compile must exit 0 with no placeholder (`sorry`/`admit`) anywhere in the file, comments included; `#print axioms` must report only the three standard axioms; and the statement comparison is re-run by an independent audit at the end. Two solution-set perturbation probes (the statement altered to a superset and to a subset of the true solution set) must turn the proof red, and they did. The verdicts for this paper are reproduced in Section 6 and ship with the artifact.

4 Main results

Theorem 2 (complete solution set, iff form)**.** *For every $n \in \mathbb{N}$, $n!! \cdot \mathrm{lcm}(1,\ldots,n)$ is a perfect square if and only if $n = 0$ or $n = 1$ or $n = 2$ or $n = 5$ or $n = 7$.*

Theorem 3 (existence-witness form)**.** *For every $n \in \mathbb{N}$, the equation $x^2 = n!! \cdot \operatorname{lcm}(1, \ldots, n)$ has a solution $x \in \mathbb{N}$ if and only if $n \in \{0, 1, 2, 5, 7\}$.*

Lemma 4 (valuation criterion)**.** *For every $n \in \mathbb{N}$: $n!! \cdot \operatorname{lcm}(1, \ldots, n)$ is a perfect square if and only if $\nu_p(n!!) + \lfloor \log_p n \rfloor$ is even for every prime p.*

Theorem 5 (solution witnesses)**.** $1^2 = 0!! \cdot \operatorname{lcm}(1, \ldots, 0)$, $1^2 = 1!! \cdot \operatorname{lcm}(1, \ldots, 1)$, $2^2 = 2!! \cdot \operatorname{lcm}(1, \ldots, 2)$, $30^2 = 5!! \cdot \operatorname{lcm}(1, \ldots, 5)$ *and* $210^2 = 7!! \cdot \operatorname{lcm}(1, \ldots, 7)$.

Listing 1: Frozen statement snapshot (Lean 4): the complete frozen set, taken from the statement source shipped as `proofs/dfact-lcm-square.statement.lean` (a comment-cleaned edition: comments carrying pre-publication workflow narrative were blanked with line numbers preserved; the statement text is byte-identical to the frozen snapshot of record, sha256 `9baa672f`; correspondence record `audit/comment-elision-record.md`).

```
theorem isSquare_dfact_mul_lcmUpto_iff (n : ℕ) :
    IsSquare (n!! * Nat.lcmUpto n) ↔
      ∀ p : ℕ, p.Prime → Even ((n!!).factorization p + p.log n)
    := by

theorem dfact_lcm_witnesses :
    1 ^ 2 = 0!! * Nat.lcmUpto 0 ∧ 1 ^ 2 = 1!! * Nat.lcmUpto 1 ∧
      2 ^ 2 = 2!! * Nat.lcmUpto 2 ∧ 30 ^ 2 = 5!! * Nat.lcmUpto 5
    ∧
        210 ^ 2 = 7!! * Nat.lcmUpto 7 := by

theorem dfact_lcm_square_solution_set (n : ℕ) :
    IsSquare (n!! * Nat.lcmUpto n) ↔ n = 0 ∨ n = 1 ∨ n = 2 ∨ n =
     5 ∨ n = 7 := by
```

5 Proofs

The final proof file (a single Lean module, 333 lines) assembles five components: the valuation criterion, the witness layer, the two exclusion arms, and the assembly of the two main theorems. The file ships byte-exactly in the artifact under `proofs/`; the excerpts below are byte-identical contiguous blocks of it (independent mechanical record: `audit/listing-verify.txt`, which ships in the deposit). The shipped file is the comment-cleaned

edition; its code is byte-identical to the frozen development (record: `audit/comment-elision-record.md`). We describe the proof in the order in which it is built.

5.1 The valuation criterion

The multiplicative spine is the product law for valuations together with the least-common-multiple factorization identity: for a prime p,

$$\nu_p\big(n!! \cdot \operatorname{lcm}(1,\dots,n)\big) = \nu_p(n!!) + \lfloor \log_p n \rfloor.$$

A natural number is a square if and only if all its prime valuations are even (`Nat.isSquare_iff_even_factorization`), so the criterion of Lemma 4 follows term by term. Listing 2 shows the whole argument: rewrite the square predicate into the even-valuation form, push the universal quantifier through the congruence, and reduce the sum to the two factors.

Listing 2: The valuation criterion (Lemma 4): the criterion bridge in full.

```
theorem isSquare_dfact_mul_lcmUpto_iff (n : ℕ) :
    IsSquare (n!! * Nat.lcmUpto n) ↔
      ∀ p : ℕ, p.Prime → Even ((n!!).factorization p + p.log n)
    := by
  rw [Nat.isSquare_iff_even_factorization]
  refine forall_congr' fun p => imp_congr_right fun hp => ?_
  simp only [Nat.factorization_mul (Nat.doubleFactorial_pos n).ne'
     (Nat.lcmUpto_ne_zero n),
    Finsupp.add_apply, Nat.factorization_lcmUpto n hp]
```

5.2 The witness layer

The five solutions are exhibited pointwise (Theorem 5, closed by decision). For the non-solutions, six explicit factorizations are recorded as separate theorems `nonsquare_at_3`, `nonsquare_at_9`, `nonsquare_at_11`, `nonsquare_at_13`, `nonsquare_at_29` and `nonsquare_at_31`: each exhibits a prime p with $\nu_p(n!! \cdot \operatorname{lcm}(1,\dots,n)) = 2e+1$ odd — for instance at $n = 3$ the value is $18 = 2^1 \cdot 9$, and at $n = 9$ it is $2381400 = 3^5 \cdot 9800$ — and the library bridge `not_isSquare_of_odd_factorization` turns an odd valuation into a non-square (a square $m = r \cdot r$ would have $\nu_p(m) = \nu_p(r) + \nu_p(r)$ even). Listing 3 shows the first of the six in full.

Listing 3: The first explicit non-square witness (at $n = 3$) and the witness layer for the five solutions.

```
theorem dfact_lcm_witnesses :
    1 ^ 2 = 0!! * Nat.lcmUpto 0 ∧ 1 ^ 2 = 1!! * Nat.lcmUpto 1 ∧
      2 ^ 2 = 2!! * Nat.lcmUpto 2 ∧ 30 ^ 2 = 5!! * Nat.lcmUpto 5
    ∧
        210 ^ 2 = 7!! * Nat.lcmUpto 7 := by
  decide

theorem nonsquare_at_3 : ¬ IsSquare (3!! * Nat.lcmUpto 3) := by
  have hM : (3!! * Nat.lcmUpto 3) = 2 ^ 1 * 9 := by decide
  apply NFactPrimorialSquare.not_isSquare_of_odd_factorization (p
    := 2) (e := 0)
    (mul_ne_zero (Nat.doubleFactorial_pos 3).ne' (Nat.
    lcmUpto_ne_zero 3))
  rw [hM, Nat.factorization_mul (by norm_num : ((2:ℕ)^1) ≠ 0) (by
     norm_num : (9:ℕ) ≠ 0),
    Finsupp.add_apply, Nat.factorization_eq_zero_of_not_dvd (by
    decide : ¬ (2:ℕ) ∣ 9),
    Nat.factorization_pow, Finsupp.smul_apply,
    Nat.Prime.factorization Nat.prime_two, Finsupp.single_eq_same
    ]; norm_num [nsmul_eq_mul]
```

5.3 The even arm

For even $n = 2m \geq 4$, Bertrand's postulate supplies a prime $p \in (m, 2m]$. Then p is odd, $p > m$, and $p \leq n < p^2$, so $\nu_p(n!!) = 0$ (the prime divides neither 2^m nor $m!$) and $\lfloor \log_p n \rfloor = 1$: an odd total. The argument is Listing 4 in full; the only library input is Bertrand's postulate and Legendre's formula in the form `Nat.factorization_factorial`.

Listing 4: The even arm in full: even $n \geq 4$ are excluded by Bertrand's postulate.

```
theorem even_dfact_lcm_nonsquare (n : ℕ) (hn : 4 ≤ n) (h2 : Even
    n) :
    ¬ IsSquare (n!! * Nat.lcmUpto n) := by
  obtain ⟨m, rfl⟩ : ∃ m, n = 2 * m := by
    obtain ⟨m, hm⟩ := h2
    exact ⟨m, by omega⟩
  have hm2 : 2 ≤ m := by omega
  obtain ⟨p, hp, hmp, hpn⟩ := Nat.bertrand m (by omega)
  have hpsq : 2 * m < p * p := by
```

```
    have h1 : 2 * m < 2 * p := by omega
    have h2' : 2 * p ≤ p * p := Nat.mul_le_mul (by omega : 2 ≤ p)
     (le_refl p)
    exact lt_of_lt_of_le h1 h2'
  have hndvd2m : ¬ p ∣ 2 ^ m := fun hd => by
    have h1 := hp.dvd_of_dvd_pow hd
    have h2 := Nat.le_of_dvd (by norm_num : (0:ℕ) < 2) h1
    omega
  have hndvdm : ¬ p ∣ m ! := fun hd => by
    have := (Nat.Prime.dvd_factorial hp).mp hd
    omega
  have hfact_dfact : ((2 * m)!!).factorization p = 0 := by
    rw [Nat.doubleFactorial_two_mul,
      Nat.factorization_eq_zero_of_not_dvd (hp.not_dvd_mul hndvd2m
     hndvdm)]
  have hlog1 : p.log (2 * m) = 1 := by
    have hge : 1 ≤ Nat.log p (2 * m) :=
      Nat.le_log_of_pow_le hp.one_lt (by rw [pow_one]; omega)
    have hlt : Nat.log p (2 * m) < 2 :=
      Nat.log_lt_of_lt_pow (by omega) (by rwa [pow_two])
    omega
  apply NFactPrimorialSquare.not_isSquare_of_odd_factorization (p
    := p) (e := 0)
    (mul_ne_zero (Nat.doubleFactorial_pos _).ne' (Nat.
    lcmUpto_ne_zero _))
  rw [Nat.factorization_mul (Nat.doubleFactorial_pos _).ne' (Nat.
    lcmUpto_ne_zero _),
    Finsupp.add_apply, hfact_dfact, Nat.factorization_lcmUpto _ hp
    , hlog1]
```

5.4 The odd arm and the interval-prime lemma

For odd n with a prime $p \in (n/4, n/3]$ and $n < p^2$, the same bookkeeping gives $\nu_p(n!!) = 2$ and $\lfloor \log_p n \rfloor = 1$, an odd total. The mechanism is the generic lemma `odd_dfact_lcm_nonsquare_of_prime`, whose statement is exactly those hypotheses; the content is Lemma 6, the existence of such a prime for all odd $n \geq 33$.

Lemma 6 (interval-prime lemma)**.** *For every $n \geq 33$ there exists a prime p with $n/4 < p \leq n/3$.*

Lemma 6 is proved in this work, not cited. Its architecture has two parts. On a bounded initial segment it is a certificate chain: a single hop

observes that a prime q with $3q \le n \le 4q-1$ witnesses $n/4 < q \le n/3$, and a chain of consecutive primes with the continuity condition $q' \le 4q/3$ covers every n from the start up to a computable bound. Beyond that bound the argument is a factorial-ratio estimate: if no prime lies in $(n/4, n/3]$ then the binomial coefficient $\binom{\lfloor n/3 \rfloor}{\lfloor n/4 \rfloor, \lfloor n/12 \rfloor}$ exceeds what the surrounding interval can supply, a contradiction. The lemma lives in an accompanying library and is consumed here by import. It is not reproduced in this paper's listings, because its proof is a separate development of the same grade; the deposit states the lemma and the architecture of its proof, and the shipped final file names the imported module. A reader who wants to re-verify the odd arm therefore needs that library as well, and this dependency is recorded in the Limitations (Section 8(b)).

5.5 Assembly

The main theorem's forward direction is by contradiction, split at $n = 4$: below the split the three small cases are decided by the witness theorems and decision procedures; at and above it, the parity of n selects the even or the odd arm, and the odd arm's remaining band $9 \le n < 33$ is closed by the six explicit witnesses interleaved with the generic odd-arm lemma at $p = 5$ and $p = 7$. The reverse direction is the witness layer. Listing 5 is the opening of that assembly.

Listing 5: The main theorem's assembly: the forward direction's case split, from the contradiction to the two arms.

```
theorem dfact_lcm_square_solution_set (n : ℕ) :
    IsSquare (n!! * Nat.lcmUpto n) ↔ n = 0 ∨ n = 1 ∨ n = 2 ∨ n =
     5 ∨ n = 7 := by
  constructor
  · intro h
    by_contra hne
    push_neg at hne
    rcases lt_or_ge n 4 with hlt | hge
    · interval_cases n
      · omega
      · omega
      · omega
```

6 Verification

The final file compiles in strict mode (errors rejected, plus a textual scan for `sorry`/`admit` including comments) with exit status 0; the only diagnostic is one style-level deprecation warning on `push_neg`. `#print axioms` on each of the twelve theorems returns exactly `[propext, Classical.choice, Quot.sound]` — reproduced verbatim in Listing 6, which is the machine output that ships in the deposit. The statement comparison against the frozen snapshot is clean declaration by declaration, and the two solution-set perturbation probes turn the proof red at exactly the perturbed theorem and its direct downstream. The independent final audit's verdict for this result is *complete proof* and ships with the artifact.

Listing 6: Axiom printout for the three frozen statements and the accompanying existential form (machine output, verbatim).

```
<source-task>/attempts/axioms-probe.lean:257:4: warning: `push_neg
    ` has been deprecated. Prefer using `push Not` instead.
If you'd rather continue using `push_neg` in your project, you can
     implement it as follows:
```
open Lean.Parser.Tactic in
macro "push_neg" cfg:optConfig loc:(location)? : tactic =>
  `(tactic| push $cfg:optConfig Not $[$loc]?)
```
'DFactLcmSquare.isSquare_dfact_mul_lcmUpto_iff' depends on axioms:
     [propext, Classical.choice, Quot.sound]
'DFactLcmSquare.dfact_lcm_witnesses' depends on axioms: [propext,
    Classical.choice, Quot.sound]
'DFactLcmSquare.nonsquare_at_3' depends on axioms: [propext,
    Classical.choice, Quot.sound]
'DFactLcmSquare.nonsquare_at_9' depends on axioms: [propext,
    Classical.choice, Quot.sound]
'DFactLcmSquare.nonsquare_at_11' depends on axioms: [propext,
    Classical.choice, Quot.sound]
'DFactLcmSquare.nonsquare_at_13' depends on axioms: [propext,
    Classical.choice, Quot.sound]
'DFactLcmSquare.nonsquare_at_29' depends on axioms: [propext,
    Classical.choice, Quot.sound]
'DFactLcmSquare.nonsquare_at_31' depends on axioms: [propext,
    Classical.choice, Quot.sound]
'DFactLcmSquare.odd_dfact_lcm_nonsquare_of_prime' depends on
    axioms: [propext, Classical.choice, Quot.sound]
'DFactLcmSquare.even_dfact_lcm_nonsquare' depends on axioms: [
```

```
    propext, Classical.choice, Quot.sound]
'DFactLcmSquare.dfact_lcm_square_solution_set' depends on axioms:
    [propext, Classical.choice, Quot.sound]
'DFactLcmSquare.exists_sq_dfact_lcm_iff' depends on axioms: [
    propext, Classical.choice, Quot.sound]
```

7 Related work and novelty

The exclusivity review's four-state adjudication is *no occupying record found*, with strength labels per channel; the full query record, including every zero hit, ships as `audit/novelty-record.md`. The channels and what they reached: (i) *OEIS value columns* — the solution-n column, the solution-root column $x = 1, 1, 2, 30, 210$, and the object value column $a(n) = n!! \cdot \mathrm{lcm}(1, \ldots, n)$ under four prefix variants (plus alternating-parity sub-columns and drop-prefix cuts in the review pass) are all null; the residual hits are unrelated items, or A061742 $= (p_n\#)^2$, which is entered precisely because $x = n\#$ and is not a registration of this equation. (ii) *arXiv metadata* — fourteen load-bearing query forms with the positive control `all:"double factorial"` returning thirty-four hits, confirming the channel was live: no same form at the metadata layer. (iii) *Neighbouring full texts* — seven items read in full, together with all seventeen references of the anchor: zero coverage of this equation. (iv) *Citation axis* — Semantic Scholar and OpenAlex, two independent channels, on the anchor: `cited_by_count` $= 0$ on both, so the anchor's open case has not been closed by anyone citing it. (v) *Public Lean ecosystem* — `doubleFactorial` occurs in exactly one file of the whole public-conjectures repository (A001818, a different object); the statement layer has zero registration. (vi) *mathlib* — ripgrep with the positive control `Nat.Prime` returning twenty-one files: `doubleFactorial` occurs in four files, none in a number-theoretic square context, `lcmUpto` in one, and the two objects never co-occur.

Three delimitations are recorded rather than left implicit. First, the $\mathrm{lcm}(1, \ldots, n)$ side of the object is occupied at the *sequence* level: OEIS A056839 registers the squarefree part of $\mathrm{lcm}(1, \ldots, n)$; that entry does not mention the double factorial anywhere, and it registers a sequence, not the characterizing solution set of this equation. Second, the review layer records that Crossref is a relevance-ranked channel and that the OpenAlex title search missed the anchor by indexing status; neither is used as a load-bearing zero hit. Third, Erdős problem #399 is *not* an open anchor — it was disproved on 2026-02-04 (counterexample $10! = 48^4 - 36^4$) — and

is mentioned only as a same-family adjacency; and A055204 asks for the *smallest* a making $n! \cdot a(n)$ a square, a different object from the present statement.

8 Limitations and threats to validity

(a) **What is proved.** The kernel layer proves exactly the three frozen statements of Listing 1 plus the proof-side additions recorded in the audit; the grade of each is *complete proof*. Nothing beyond is claimed: no growth statement, no other multiplier in place of $\mathrm{lcm}(1, \ldots, n)$, and no first case.

(b) **The interval-prime lemma is proved here, not cited.** Lemma 6 is a certificate chain over a bounded segment plus a factorial-ratio tail estimate, both built in the accompanying library development rather than taken from a published interval theorem. The paper makes no claim that it is an instance of, or comparable in strength to, any published interval theorem; in particular it is not presented as an application of Nagura's bound. The corresponding disclosure on the reader's side: this deposit ships the final development, which imports that lemma module, but not the module itself, so the odd arm cannot be re-verified from the deposited files alone. Both the odd-arm mechanism and one bridge used by the witness layer are library items, each with its own separately checked final state; their grades are inherited here, not re-established.

(c) **The first case is not claimed.** The companion equation $x^2 = n! \cdot n\#$ of the same remark in [1] is not claimed here; it was settled separately [9].

(d) **No registration claim.** No OEIS registration is claimed or performed; the review layer found no registration of the value columns, and any registration action is the author's own.

(e) **An observation, not a theorem.** The observation that the five solution roots are $x = n\#$ (the primorials $0\#, 1\#, 2\#, 5\#, 7\# = 1, 1, 2, 30, 210$) is a numerical observation recorded alongside the theorem; it was not formalized as a statement, and the paper does not present it as a proved proposition.

(f) **A numerical coincidence, explicitly disclosed.** The intersection $\{5, 7\}$ of this solution set with the Brocard solutions $\{4, 5, 7\}$ is a numerical coincidence; this result implies nothing in either direction about the Brocard equation, and no Brocard-direction conclusion is claimed.

(g) **Comment-level deltas.** The shipped `.lean` files are copies in which comments carrying pre-publication workflow narrative were blanked (line

numbers preserved, code bytes untouched); per file the comment-free code view of the source and of the copy are byte-for-byte equal, asserted by the generating script. Statements and proof terms are therefore exactly the frozen artifacts, and the compile and axiom records shipped in `audit/` were produced from the sources in their original, unblanked form.

(h) **Semantic fidelity of the formalization.** The theorems are about the Lean definitions. Their fidelity to the informal reading rests on the frozen statements being the intended encoding; a semantic round-trip check (formal statement back to natural language, adjudicated consistent) was performed at the formalization stage, and a further spot check of this paper's informal exposition against the frozen statements was done at assembly. A reader who disagrees with the encoding is disagreeing with the statement, not with the proof.

(i) **Novelty review limits.** The zero-hit verdict is a statement about the channels reached (arXiv metadata, OEIS keyword and value faces, seven neighbouring full texts, two citation channels, the public Lean ecosystem via a repository tarball, the local mathlib tree), not about the literature as a whole; background items recorded as undecided at the full-text level stay in the source task card, which is not shipped, and none of them is load-bearing for this result. No "first formalization" claim is made.

(j) **Numeric evidence is layered.** The factorization witnesses quoted in the text are numeric-level evidence; every theorem-level conclusion is adjudicated by the kernel and depends on no numeric evidence. The $[0, 64]$ squareness scan reported by the audit is corroboration, not a premise.

Data availability

Lean sources, the frozen statement, compile and axiom records, and the full novelty-search record are packaged on Zenodo (DOI: `10.5281/zenodo.23309984`). The paper is released under CC BY 4.0 and the code under MIT. A companion claim of record for the same kernel result is deposited separately.

Use of generative AI

The formalization and proofs reported in this paper were developed with AI assistance (large language model: `kimi-k3`) under human direction and review. All formal statements and proofs reported in this paper were checked by the Lean 4 kernel (Lean v4.34.0, mathlib v4.34.0): every proof compiles

with no `sorry`, and `#print axioms` reports only {propext, Quot.sound, Classical.choice}. The author reviewed all content and takes full responsibility for it.

Author contributions

The author selected the problem, verified the semantics of the formal statements against the informal ones, and reviewed the kernel evidence and the novelty record before signing off the deposit.

References

[1] S. Novaković. The Diophantine equation $P(x) = \prod_{i=1}^{r} H_{n_i}$. arXiv:2601.16757, 2026.

[2] S. Novaković. Diophantine equations involving double factorials. arXiv:2510.24312, 2025.

[3] S. Novaković. On a generalization of the Brocard–Ramanujan Diophantine equation. arXiv:2602.09687, 2026.

[4] S. Novaković. Products of factorials which are products of factorials. arXiv:2602.23838, 2026.

[5] S. Novaković. A comment on the equation $n!! = a_1!! \cdots a_t!!$. arXiv:2604.09730, 2026.

[6] F. Yudin. Square products of factorials and a conjecture of Erdős and Graham. arXiv:2610.01899, 2026.

[7] L. V. Peixoto. The Brocard Problem: Structural Invariants, p-Adic Density, and the Generative Sieve. arXiv:2606.23485, 2026.

[8] Erdős problem #399, erdosproblems.com. Recorded as disproved (2026-02-04); mentioned here only as a same-family adjacency.

[9] A. Aurora0134. The Diophantine equation $x^2 = n! \cdot n\#$: the complete solution set, formalized and machine-checked in Lean 4. Zenodo deposit, 2026. `https://doi.org/10.5281/zenodo.23179326`

[10] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *CADE-28*, 2021. https://doi.org/10.1007/978-3-030-79876-5_37

[11] The mathlib Community. The Lean mathematical library. *CPP 2020*, 2020. https://doi.org/10.1145/3372885.3373824